

Methodology

1.1 Introduction

Transformer encoders such as BERT remain attractive for text classification because they offer strong accuracy, stable training behaviour, and simple deployment interfaces. Their practical weakness is that a standard finetuned encoder is often much larger than the downstream task requires. The dense model spends compute on every attention head and every feed forward neuron at every layer, even when a large fraction of those structures can be removed without destroying the task solution. This report studies how to choose those structures under a strict inference budget and how to recover the accuracy of the resulting sparse architecture without retraining a new model from scratch.

The compression setting considered here is deliberately structured. The method removes complete attention heads and complete feed forward neurons, and in some cases removes an entire transformer layer by making it an identity map. This is different from unstructured weight pruning, where individual scalar weights are zeroed. The reason is operational rather than aesthetic. A dense smaller matrix is easy to export, run, profile, and compare; an irregular sparse matrix requires runtime support that may not exist on the deployment target and often fails to deliver a proportional speedup. The goal of the report is therefore not merely to reduce the number of non zero parameters, but to construct architectures whose forward pass is genuinely cheaper in the dimensions that matter for inference.

The central difficulty is that the architecture selection problem is combinatorial. A BERT-base encoder has twelve layers, twelve heads per layer, and 3072 feed forward neurons per layer. A candidate architecture is a binary decision over all of these structures. Exhaustive enumeration is impossible, and direct evaluation of a candidate by fully training or fully recovering it is too expensive to use inside a search loop. The method therefore separates the problem into two phases. The first phase searches over binary architectures using a cheap calibration metric. The second phase takes the selected architecture and performs a multi-stage recovery procedure based on distillation from the finetuned teacher.

The search phase is not a continuous relaxation of the masks. It is a direct search over hard binary masks. This choice avoids the gap between a relaxed gate and the binary architecture eventually deployed, but it also means that the search needs a robust sampling procedure. The report uses an evolutionary algorithm with a hard compute budget. The budget is expressed as a fixed operation count at a reference sequence length, and candidates are repaired into a narrow feasibility band around that budget. This constraint is essential: without it, a teacher-matching surrogate naturally favours keeping more structure, and the search degenerates into a size comparison rather than an allocation comparison. Inside the budget band, candidates have similar cost and differ mainly in where compute is placed across depth, attention, and feed forward capacity.

The surrogate metric used during search compares the masked student and the dense teacher on a calibration subset. It uses representation and prediction signals that are cheap enough to compute for many candidates. The report treats this metric as a practical instrument, not as an oracle. Several experiments show that the metric can mis-rank architectures, especially at high sparsity, because the unadapted masked student is not the same object as the recovered student. For that reason, the search includes diversity preservation, candidate caching, multi-fidelity

evaluation, and a final funnel rather than trusting a single scalar score blindly.

The recovery phase is equally important. A selected mask can have very low raw accuracy before adaptation. The recovery procedure first hardens the masks and establishes the masked model state, then uses intermediate representation distillation to rebuild the internal trajectory of the student, and finally uses prediction distillation and task loss to improve the classifier-facing behaviour. The distinction between these stages matters empirically. On QNLI, for example, the representation recovery stage accounts for most of the accuracy restoration, while later prediction finetuning provides a smaller final correction. The report therefore describes the recovery procedure in detail rather than treating it as a generic finetuning step.

The contribution of the work is the full system: a hard-budget structured search over heads and neurons, a calibration metric designed for fast candidate evaluation, an evolutionary search procedure with diversity and caching machinery, and a multi-stage recovery method that makes the selected architecture usable. The experimental sections evaluate the resulting method on GLUE classification tasks, compare against simpler search baselines, ablate the search and recovery components, and diagnose where the metric succeeds or fails as a predictor of post-recovery accuracy.

The remainder of this chapter is organised as follows. Sections 1.2 through 1.5 define the candidate representation, the cost model, and the calibration signals. Sections 1.4 through 1.9 describe the surrogate metric and the search algorithm. Section 1.10 describes the recovery procedure. The final sections report the experimental protocol and conclusions.

1.2 Mask Parameterisation

This section defines the architecture variable optimised by the search. Each candidate is represented by two binary masks: one over attention heads and one over feed forward neurons. The masks do not change the embedding dimension, classifier interface, residual topology, or layer ordering. They only decide which structured units remain active, matching the unit level used by structured transformer pruning methods (Michel et al., 2019; Xia et al., 2022; Kurtic et al., 2023).

1.2.1 Candidate Space

A candidate architecture is a pair of binary masks

$$c = (\mathbf{M}^H, \mathbf{M}^N) \in \{0, 1\}^{L \times H} \times \{0, 1\}^{L \times N}. \quad (1.1)$$

For layer ℓ , the active attention heads and active feed forward neurons are

$$H_\ell(c) = \{h : \mathbf{M}^H[\ell, h] = 1\}, \quad N_\ell(c) = \{n : \mathbf{M}^N[\ell, n] = 1\}. \quad (1.2)$$

The head mask selects complete head slices in the attention projections. The neuron mask selects rows of the first feed forward projection and the matching columns of the second feed forward projection. The masks are indexed in the original teacher coordinates.

The remaining definitions are derived from this same object: structural counts, layer status, and candidate identity.

1.2.2 Structural Counts

The first derived object is the count vector induced by the two masks. The count vector summarizes the mask for budget control and reporting. At layer ℓ , summing the binary decisions

in each row gives

$$a_\ell(c) = \sum_{h=1}^H \mathbf{M}^H[\ell, h], \quad (1.3)$$

$$n_\ell(c) = \sum_{n=1}^N \mathbf{M}^N[\ell, n]. \quad (1.4)$$

Thus $a_\ell(c)$ records how many attention heads remain in layer ℓ , whereas $n_\ell(c)$ records how many feed forward neurons remain in that same layer. The search keeps these local quantities because cost and behaviour depend on where units are kept.

The global head and neuron counts are then obtained by summing the local counts across depth:

$$A(c) = \sum_{\ell=1}^L a_\ell(c), \quad N(c) = \sum_{\ell=1}^L n_\ell(c). \quad (1.5)$$

These totals report candidate size. They do not show whether the remaining units are concentrated in a few layers or distributed across many layers. Therefore, we also track the number of layers that contain at least one active sublayer:

$$L_{\text{act}}(c) = \sum_{\ell=1}^L \mathbf{1}\{a_\ell(c) + n_\ell(c) > 0\}. \quad (1.6)$$

Finally, the same per-layer counts are used to evaluate the compute budget. Each active head contributes one head cost, and each active feed forward neuron contributes one neuron cost:

$$\text{MAC}(c) = \sum_{\ell=1}^L (a_\ell(c)c_H + n_\ell(c)c_N), \quad (1.7)$$

where c_H and c_N are the per head and per neuron costs derived in Section 1.3.

The pair (a_ℓ, n_ℓ) also fixes the execution case of layer ℓ . Table 1.1 shows how these counts determine the masked forward pass. The search therefore selects both the budget and the computation that remains at each depth.

Condition	Status	Masked forward
$a_\ell > 0, n_\ell > 0$	both active	attention and feed forward execute
$a_\ell = 0, n_\ell > 0$	feed forward only	attention update is zero; feed forward executes
$a_\ell > 0, n_\ell = 0$	attention only	attention executes; feed forward update is zero
$a_\ell = 0, n_\ell = 0$	identity	the layer returns its input

Table 1.1: Layer statuses induced by the structural masks.

This status is used whenever a candidate is evaluated. Positive counts keep the corresponding sublayer active, whereas zero counts remove that sublayer update. Therefore, two candidates with the same global size can behave differently if their active units create different layer statuses. The next step is to define how the two mask types interact inside an active layer.

1.2.3 Why Heads and Neurons Are a Joint Choice

The search represents $\mathbf{M}^H$ and $\mathbf{M}^N$ as a single candidate. A head decision changes the residual stream before the feed forward block, and a neuron decision changes how that stream is

transformed. Let x_ℓ be the input to layer ℓ . With active head set H_ℓ and active neuron set N_ℓ , the masked layer can be written abstractly as

$$u_\ell = \text{LN}_\ell^A(x_\ell + A_\ell(x_\ell; H_\ell)), \quad (1.8)$$

$$x_{\ell+1} = \text{LN}_\ell^F(u_\ell + F_\ell(u_\ell; N_\ell)). \quad (1.9)$$

Here A_ℓ is the attention update restricted to the surviving heads, and F_ℓ is the feed forward update restricted to the surviving neurons. Because F_ℓ is evaluated at u_ℓ , changing H_ℓ changes the input seen by the surviving feed forward neurons. A mutation to one mask can therefore change the effect of the units selected by the other mask. The search scores, caches, and mutates the pair jointly.

After this coupling is fixed, the implementation needs two levels of candidate description. The exact description prevents repeated evaluations of the same mask. The coarser description tracks the shape of the population.

1.2.4 Granularity and Candidate Identity

Neuron choices are grouped for search efficiency. The implementation allocates feed forward neurons in blocks of sixteen. Thus $n_\ell(c)$ is restricted to multiples of sixteen, except for the zero case. This block size keeps mutation aligned with meaningful cost changes and reduces candidates that differ only by negligible single-neuron moves.

For exact caching, the identity of a candidate is the bit string obtained by concatenating $\mathbf{M}^H$ and $\mathbf{M}^N$. Two candidates are identical if and only if this bit string is identical. This key is deliberately fine-grained because it preserves the exact placement of every active unit. For diversity tracking, the search also records the coarser macro signature

$$\sigma(c) = (A(c), N(c), L_{\text{act}}(c)). \quad (1.10)$$

The macro signature loses placement information. Two candidates can have the same total counts and active layer count while assigning compute to different depths, so exact deduplication still uses the full bit string. The macro signature is used only as a population-level descriptor in the island and diversity mechanisms of Section 1.8.

1.3 Efficiency Budget and Cost Model

The mask defined in Section 1.2 deactivates a subset of attention heads and feed-forward neurons in each layer. Since the active heads and neurons are fixed by the mask, the computational cost of each candidate is determined directly by that mask. The remaining components form a smaller sub-network whose forward pass evaluates fewer active parameters than the dense model. To compare candidates under a fixed compute target, we express this cost as a MAC count derived from the mask. This value is used consistently for feasibility checks during search, repair of infeasible offspring, and final reporting. All costs are measured at the reference sequence length S used for calibration, so every candidate is compared under the same input shape.

The following subsections define this MAC count from the active attention heads and feed-forward neurons that remain after pruning. We first compute the cost of one active head and one active feed-forward neuron, then aggregate these costs within each layer and across the encoder.

1.3.1 Per Unit Costs

The unit costs follow directly from the linear projections that survive under the mask. For one active attention head with head dimension d_h , the input and output projections give the first order term

$$c_H = 2Sdd_h, \quad (1.11)$$

where the contributions that do not depend on the head mask are absorbed into a constant and dropped. For one active feed forward neuron the two linear maps give

$$c_N = 2Sd, \quad (1.12)$$

the first selecting the neuron from the residual stream and the second projecting it back. Both unit costs are independent of layer index, so the cost of a layer is determined by the number of active heads and active neurons it contains.

1.3.2 Candidate Cost

Let $a_\ell(c)$ and $n_\ell(c)$ denote the active head and neuron counts of layer ℓ under the mask of candidate c . The layer cost is the linear combination

$$\text{MAC}_\ell(c) = a_\ell(c)c_H + n_\ell(c)c_N, \quad (1.13)$$

and the candidate cost is the sum across depth,

$$\text{MAC}(c) = \sum_{\ell=1}^L \text{MAC}_\ell(c). \quad (1.14)$$

The dense reference $\text{MAC}_{\text{dense}} = L(Hc_H + Nc_N)$ gives the cost of the unmasked model, while $\rho(c) = \text{MAC}(c)/\text{MAC}_{\text{dense}}$ reports each candidate as a fraction of dense cost. This global value does not replace the layer-wise decomposition, this implies that the global cost alone is not sufficient: two masks can have equal cost while distributing compute across different layers. The surrogate of Section 1.4 therefore depends on the placement of compute, not only on its total amount.

1.3.3 Example Decomposition

We instantiate the cost model on BERT-base, with $L = 12$, $H = 12$, $d = 768$, $d_h = 64$, $N = 3072$, and reference length $S = 128$. Substituting into (1.11) and (1.12) gives the unit costs; the layer and total costs of the illustrative candidate of Table 1.2 follow from (1.13) and (1.14).

Table 1.2: Per layer decomposition. Unit costs from (1.11)–(1.12):
Costs in M ops.

ℓ	a_ℓ	n_ℓ	Head	Neur	Tot
0	2	112	25	22	47
1	3	96	38	19	57
2	2	64	25	13	38
3	2	64	25	13	38
4	0	16	0	3	3
5	0	16	0	3	3
6	0	0	0	0	0
7	1	32	13	6	19
8	0	32	0	6	6
9	1	32	13	6	19
10	0	16	0	3	3
11	1	16	13	3	16
Σ	12	496	152	97	249

$$c_H = 2 \cdot 128 \cdot 768 \cdot 64 \approx 12.58 \text{ M},$$

$$c_N = 2 \cdot 128 \cdot 768 \approx 0.197 \text{ M}.$$

Layer 0 ($a_\ell = 2$, $n_\ell = 112$):

$$\text{MAC}_0 = 2c_H + 112c_N \approx 47.2 \text{ M}.$$

Candidate total and dense reference:

$$\text{MAC}(c) \approx 249 \text{ M}, \quad \text{MAC}_{\text{dense}} \approx 9060 \text{ M}.$$

Keep ratio:

$$\rho(c) = 249/9060 \approx 0.027.$$

The candidate retains roughly 2.7% of dense cost; compute is concentrated in the lower layers and the middle is nearly emptied.

1.3.4 Feasibility Band

The target budget B is not generally reachable exactly. Head counts are integers and neuron counts are allocated in blocks of sixteen, so the set of attainable values of $\text{MAC}(c)$ is discrete and the chance that any single point coincides with B is small. The search therefore accepts a candidate as feasible whenever its cost lies in a narrow band around the target,

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B, \quad (1.15)$$

which keeps candidates at a comparable compute level and reduces the comparison between them to the question of where the budget is placed, not how much of it is spent. The half width ε is a hyperparameter of the search, chosen small enough that the band does not confound compute reductions with metric improvements and large enough that repair operators have a non empty target set. Operation count is used for the constraint because it is deterministic and device independent; a deployment specific latency table can replace it without affecting the rest of the procedure.

The budget model therefore fixes the size of the candidate and leaves the allocation problem open. Several masks can satisfy the same operation band while placing their heads and neurons at very different depths. The next section defines the surrogate metric used to choose among these equal-cost candidates before any recovery training is performed.

1.4 The Surrogate Distance Metric

After budget filtering, the candidates being scored are already close in operation count. The remaining question is allocation. The same budget can be placed at different depths and split differently across the two structural axes. A useful search metric therefore has to compare how these allocations behave, since the cost alone no longer separates them.

The metric is evaluated before recovery training. This restriction is essential because the search scores thousands of masks, and running a full distillation or fine tuning procedure for every candidate would make the search infeasible. The metric therefore has to judge an unrecovered masked model from a small calibration pack. Validation accuracy and final task loss are weak by themselves at this stage because high-sparsity candidates often produce flat or collapsed outputs.

The implementation uses two ranking steps. The first step ranks candidates by residual-update preservation and keeps the best fraction of the cohort. The second step ranks the survivors by a task-facing distance. This order reflects the role of the metric in the search: a candidate must first preserve a plausible computation path, then the task signal decides which preserved path is already closest to the teacher’s prediction surface.

1.4.1 Equal-Cost Candidate Ranking

The budget constraint turns a large architecture space into a local comparison problem. The metric receives a cohort of feasible allocations near the target cost, so the candidates differ mainly in where their remaining units are placed.

Let $\mathcal{C}_t$ be the cohort evaluated at generation t . Every $c \in \mathcal{C}_t$ satisfies the feasibility band

$$(1 - \varepsilon)B \leq \text{MAC}(c) \leq (1 + \varepsilon)B. \quad (1.16)$$

The band makes the members of $\mathcal{C}_t$ comparable in cost. Small residual MAC differences inside the band are treated as secondary; the main comparison is the placement of the retained heads and feed forward neurons. This is why the score is cohort-local: it decides which feasible allocation should survive this generation.

For each candidate, the teacher $\mathcal{T}$ and masked student $\mathcal{S}(c)$ are evaluated on a calibration pack

$$\mathcal{D}_{\text{cal}} = \{x_i\}_{i=1}^B.$$

Let $p_{i,s} \in \{0, 1\}$ indicate whether token position s of example i is non-padding. All hidden-state and update distances are computed only over positions where $p_{i,s} = 1$.

The output of the metric is a cohort-local ordering. Lower values are better:

$$\Phi(c_1) < \Phi(c_2) \implies c_1 \text{ is preferred to } c_2 \text{ inside the current search generation.}$$

This ordering drives search-time selection before recovery. The first quantity needed by the two-step rule is the structural rank, so the metric starts from the layer updates.

1.4.2 Residual Update Matching

The structural rank is based on a simple intuition: a retained architecture should reproduce the transformations made along the teacher’s depth. Comparing final hidden states is too coarse for this purpose. A hidden state contains the accumulated residual stream, so a weak layer can be hidden by computation that happened earlier. The layer update exposes the contribution made at one depth.

Let h_ℓ denote the hidden state after layer ℓ , with h_0 the embedding output. The layer update is

$$\Delta h_\ell = h_\ell - h_{\ell-1}. \quad (1.17)$$

For the teacher and student, these updates are written as $\Delta_\ell^{(\mathcal{T})}$ and $\Delta_\ell^{(\mathcal{S})}$.

The update Δh_ℓ isolates the computation performed at layer ℓ . If a masked layer contributes almost nothing, then $\Delta_\ell^{(\mathcal{S})}$ becomes small even when the absolute hidden state remains numerically close to the teacher stream. This makes update matching a better search signal for partial and fully dropped layers.

For tensors A and B shaped as batch, sequence, and hidden dimension, define the calibration inner product

$$\langle A, B \rangle_{\text{cal}} = \sum_{i=1}^B \sum_{s=1}^S p_{i,s} A_{i,s}^\top B_{i,s}, \quad \|A\|_{\text{cal}} = \sqrt{\langle A, A \rangle_{\text{cal}}}. \quad (1.18)$$

The inner product sums only real token positions, so padding does not change the distance. The per-layer trajectory distance then checks two properties of the student update. Direction agreement asks whether the student update points the same way as the teacher update. Projection agreement asks whether the student recovers the teacher update with the right scale along that direction:

$$d_{\ell, \text{cos}}^{\text{traj}}(c) = \frac{1}{\pi} \arccos \left(\frac{\langle \Delta_\ell^{(\mathcal{S})}, \Delta_\ell^{(\mathcal{T})} \rangle_{\text{cal}}}{\|\Delta_\ell^{(\mathcal{S})}\|_{\text{cal}} \|\Delta_\ell^{(\mathcal{T})}\|_{\text{cal}} + \delta} \right), \quad (1.19)$$

$$\alpha_\ell(c) = \frac{\langle \Delta_\ell^{(\mathcal{S})}, \Delta_\ell^{(\mathcal{T})} \rangle_{\text{cal}}}{\|\Delta_\ell^{(\mathcal{T})}\|_{\text{cal}}^2 + \delta}, \quad (1.20)$$

$$d_{\ell, \text{proj}}^{\text{traj}}(c) = |1 - \alpha_\ell(c)|, \quad (1.21)$$

$$d_\ell^{\text{traj}}(c) = (1 - \lambda) d_{\ell, \text{cos}}^{\text{traj}}(c) + \lambda d_{\ell, \text{proj}}^{\text{traj}}(c). \quad (1.22)$$

The constant δ is a numerical stabiliser. The cosine term measures direction agreement between the student and teacher updates. The projection coefficient α_ℓ measures how much of the teacher update is recovered by the student along the teacher direction. A value near one means the student update has the correct magnitude in the teacher direction. A value near zero means the update is missing or orthogonal. Values above one mean the student overshoots the teacher update.

Both parts are needed. A layer with the right norm and wrong direction fails the cosine term. A layer with the right direction and weak magnitude fails the projection term. A dropped layer gives $\Delta_\ell^{(S)} \approx 0$, so its projection coefficient is close to zero. The per-layer distance therefore penalises missing, misdirected, and over-scaled computation.

1.4.3 Aggregation Across Layers

A candidate is selected as a whole architecture, so the layer distances need a single structural score. The aggregation below turns the sequence of per-layer errors into one trajectory distance:

$$d^{\text{traj}}(c) = \frac{1}{Z} \sum_{\ell=1}^L w_\ell d_\ell^{\text{traj}}(c), \quad Z = \sum_{\ell=1}^L w_\ell. \quad (1.23)$$

The weights w_ℓ determine how depth contributes to the structural score. The simplest choice is $w_\ell = 1$, which gives each layer equal influence. A magnitude-weighted variant uses a function of $\|\Delta_\ell^{(T)}\|_{\text{cal}}$, so layers where the teacher update is larger receive more weight. A late-layer-weighted variant can emphasise depths closer to the classifier. These choices change the search pressure, so the experiments report them as metric ablations.

The trajectory score is the first-stage rejector in the metric. Candidates with many missing, misdirected, or scale-wrong updates receive high d^{traj} . Candidates that preserve a coherent computation path receive lower d^{traj} . At this point the metric has identified which masks still perform a plausible internal computation; the next stage can then use final-output evidence to order those masks.

1.4.4 Task-Facing Distance After Structural Filtering

The structural score does not inspect the final label decision. The second ranking stage fills that gap with task-facing evidence from the final logits. This signal is useful after the update gate, because the surviving candidates have already preserved enough internal computation to make their logits meaningful.

Let $z_i^{(T)}$ and $z_i^{(S)}(c)$ be the teacher and student logits for input x_i . The distillation distance is

$$d^{\text{kd}}(c) = T^2 \frac{1}{B} \sum_{i=1}^B \text{KL} \left(\text{softmax}(z_i^{(T)}/T) \parallel \text{softmax}(z_i^{(S)}(c)/T) \right), \quad (1.24)$$

where T is the distillation temperature. The hard task-facing term is

$$d^{\text{ce}}(c) = -\frac{1}{B} \sum_{i=1}^B \log \text{softmax}(z_i^{(S)}(c))_{y_i^*}, \quad (1.25)$$

where y_i^* is either the true label or the teacher argmax label, depending on the calibration mode.

At high sparsity, many unrecovered candidates produce logits that are nearly uniform, biased toward one class, or unstable under small mask changes. In that regime, d^{kd} and d^{ce} can be similar for masks with different internal computations. This is the reason the search does not use the task distance as the first gate. The task distance is written as

$$d^{\text{task}}(c) = \beta_{\text{kd}} d^{\text{kd}}(c) + \beta_{\text{ce}} d^{\text{ce}}(c), \quad (1.26)$$

with non-negative coefficients. This distance is computed for every candidate in the cohort, then converted to a percentile rank. The final selection rule uses this task rank after the update-rank gate has selected the structurally acceptable candidates.

A low task distance from a collapsed candidate can occur for accidental reasons on a small calibration pack. A low task distance from a candidate with good residual updates is more meaningful, since the candidate has preserved a computation path capable of supporting recovery.

1.4.5 Two-Step Cohort Ranking

The search implementation uses the double-rank gate in `updt_gate` mode. This is the point where the two distances become the actual selection score. Raw trajectory and task distances have different units, ranges, and noise levels, so the implementation compares candidates by percentile ranks within the current cohort.

For any component d , define the lower-is-better percentile rank

$$r_d(c) = \frac{1}{\max(|\mathcal{C}_t| - 1, 1)} \text{rank}_{\mathcal{C}_t}(d(c)), \quad (1.27)$$

where ties receive their averaged rank and lower values are better. The two ranks used by the search are

$$r^{\text{traj}}(c) = r_{d^{\text{traj}}}(c), \quad r^{\text{task}}(c) = r_{d^{\text{task}}}(c).$$

Here r^{traj} comes from the trajectory distance and r^{task} comes from the task-facing distance. The ranking rule then uses them in a fixed order.

The first ranking step keeps candidates whose update rank is inside the retained fraction ρ :

$$\mathcal{S}_t = \{c \in \mathcal{C}_t : r^{\text{traj}}(c) \leq \rho\}, \quad \rho = \text{UPDT_GATE_KEEP_FRAC}.$$

This step implements the structural filter: candidates outside $\mathcal{S}_t$ have weaker update preservation than the retained fraction of the cohort. The second step orders the survivors by task rank, with a small update-rank tie-break:

$$\Phi(c) = \begin{cases} r^{\text{task}}(c) + \epsilon r^{\text{traj}}(c), & c \in \mathcal{S}_t, \\ 1 + r^{\text{traj}}(c), & c \notin \mathcal{S}_t. \end{cases} \quad (1.28)$$

The first branch is the survivor ordering used for selection. The second branch assigns every rejected candidate a score above one, placing it behind the survivor set. Rejected candidates still keep their update-rank order, so the genetic search retains pressure toward better structure even among candidates that fail the current gate.

1.4.6 Calibration Noise and Resampling

The two-step ranking depends on measurements from a finite calibration pack. A candidate near the update gate can change rank if the pack changes. The implementation reduces this variance by averaging component distances over independent packs before computing ranks. Let $\mathcal{P}_1, \dots, \mathcal{P}_R$ be independently sampled calibration packs. A resampled component is computed as

$$\bar{d}(c) = \frac{1}{R} \sum_{r=1}^R d_{\mathcal{P}_r}(c). \quad (1.29)$$

The same averaging applies to d^{traj} , d^{kd} , and d^{ce} before cohort ranks are computed.

The search uses two fidelity levels. Cheap evaluation scores many candidates on a small pack and drives broad exploration. Full evaluation averages over larger or multiple packs for candidates that may survive, enter the Hall of Fame, or be compared across generations. The cheap score gives speed. The full score reduces rank flips near the structural gate.

Resampling is especially important around κ . A small calibration perturbation can move a candidate from survivor to non-survivor. Averaging reduces these boundary errors and makes the selected population less dependent on one calibration draw.

1.4.7 Limits of the Surrogate

The surrogate ranks unrecovered candidates. It cannot guarantee recovered validation accuracy. Recovery can repair a candidate with a mediocre initial surrogate score, and recovery can fail on a candidate whose initial updates look good. The metric is therefore evaluated empirically by rank correlation with post-recovery validation accuracy in the later diagnostic experiments.

1.5 Calibration Data and Signal Collection

The search metric in Section 1.4 is evaluated many times, so it cannot use the full training or validation set for every candidate. It uses a small calibration subset instead. This subset supplies the inputs on which the teacher and the masked student are compared during search.

1.5.1 Calibration Subset

The calibration subset is sampled from the training split with a fixed seed. The validation split is reserved for reporting recovered accuracy, so it is not used as the search signal. This keeps the search target separate from the final evaluation target.

Each calibration example is tokenised in the same way as the task data used for training and recovery. Padding positions are tracked, because hidden-state and update distances are computed only on real tokens. Labels are kept for the task-facing terms of the surrogate, such as cross entropy, but the calibration subset is not used to update model parameters during search.

1.5.2 Cached Teacher Signals

The teacher is fixed while candidate masks are being searched. Its outputs on the calibration subset are therefore computed once and reused. The cached signals are the teacher logits and the teacher hidden states at each layer.

The logits support the task-facing part of the metric. The hidden states support the residual-update distance, since layer updates are computed from consecutive hidden states. Caching these teacher signals removes the teacher forward pass from each candidate evaluation. Candidate scoring then requires forwarding only the masked student on the same calibration inputs and comparing its outputs to the cached teacher signals.

1.5.3 Cheap and Full Packs

The implementation uses two calibration fidelities. A cheap pack contains fewer calibration tokens and is used when many candidates must be screened quickly. A full pack contains more tokens and is used for periodic full evaluations, Hall-of-Fame candidates, and final reranking.

Both packs are built from the same calibration subset. The cheap pack gives the search enough signal to discard weak candidates at low cost. The full pack reduces noise when a candidate is important enough to affect the population or the final selected architecture.

For full evaluation, the implementation can average over multiple independently sampled full packs. This resampling is used before cohort ranks are computed, so the update-rank gate and the task-rank ordering depend on averaged component distances rather than on a single calibration draw.

1.6 The Genetic Search Algorithm

The preceding sections have fixed everything the search operates on: a candidate $c = (\mathbf{M}^H, \{n_\ell\}_{\ell=1}^L)$ is the binary head mask and neuron-count vector of Section 1.2, the feasibility band $\mathcal{X}_B = \{c : (1-\varepsilon)B \leq \text{MAC}(c) \leq (1+\varepsilon)B\}$ comes from the cost model of Section 1.3, and candidates are

compared on the calibration packs $\mathcal{D}_{\text{cal}}^{\text{cheap}}, \mathcal{D}_{\text{cal}}^{\text{full}}$ of Section 1.5 by the surrogate Φ of Section 1.4. What remains is to choose one $c \in \mathcal{X}_B$. The space $\mathcal{X}_B$ is discrete, combinatorial in the per-layer head and neuron choices, and structured: feasible neighbours obtained by a single budget-neutral swap tend to share most of their surrogate score, while large structural moves jump across plateaus. We approach the problem with an evolutionary procedure: it proposes masks, repairs them into $\mathcal{X}_B$, scores them on the calibration packs, and keeps the ones whose layerwise residual updates and task-facing signals rank well within a cohort, while preserving executable structural cases and a floor $N_a(c) \geq L_{\min}$ on active depth. Formally the procedure implements the inexact minimiser

$$\hat{c} \in \arg \min_{c \in \mathcal{X}_B} \Phi(c). \quad (1.30)$$

The rest of this section defines each component of that procedure, starting with the high-level pipeline and then descending into the math of each block.

1.6.1 Overview

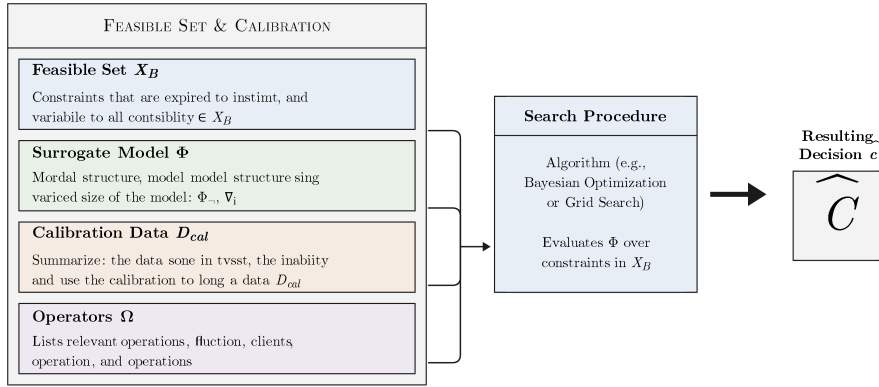


Figure 1: The main inputs are ir feasible between the feasible Set $\mathcal{X}_B$ in λ . The Surrogate Model Φ is strugel-line comiincants and operation anulat3sand relevant constraintsin Ω .

Figure 2: SearchProcedure Algorithm. c . Evaluates Φ over constraintsin $\mathcal{X}_B$.

Final Resulting Decision c

Figure 1.1: Inputs and output of the search. The four objects fixed by Sections 1.2–1.4 are passed to the search procedure, which returns a single mask $\hat{c}$.

The search produces $\hat{c}$ through an iterated sequence of calls to the surrogate Φ on candidates drawn from $\mathcal{X}_B$. Each candidate carries a binary head mask and a vector of neuron counts; the budget cost $\text{MAC}(c)$ is determined by those choices through the operation model of Section 1.3; the surrogate value $\Phi(c)$ is determined by them through the metric of Section 1.4 applied to the calibration packs of Section 1.5. Two side conditions hold throughout: every mask must correspond to an executable forward pass, and the active-depth floor $N_a(c) \geq L_{\min}$ prevents the architecture from collapsing into too few layers. The total compute budget of the search is set by a target number of surrogate calls; every component below is designed under that single budget.

The feasibility band $\mathcal{X}_B$ has a two-scale structure that the procedure exploits. Feasible neighbours obtained by a single budget-neutral swap of a head against a block of neurons, or of one head against another, differ in their surrogate score by a small amount; widely separated masks, such as one that retains the lower half of the encoder and one that retains the upper half at the same operation count, differ qualitatively in both their representations and their recoverability. A search that allocates its surrogate calls only at the first scale converges quickly to a local minimum that depends on its starting point; a search that allocates only at the second scale fails to refine a chosen macroscopic shape. The procedure carries both: local moves within

a population act at the first scale, and several populations placed in different regions of $\mathcal{X}_B$ act at the second.

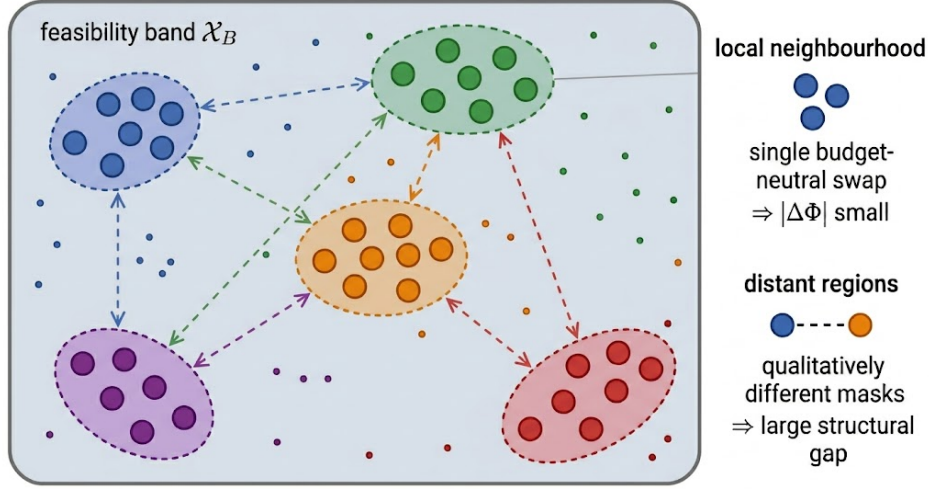


Figure 1.2: The feasibility band $\mathcal{X}_B$ as a populated space. Feasible candidates form clusters of architecturally similar masks; clusters in different regions of the band differ qualitatively.

To explore different regions of $\mathcal{X}_B$ simultaneously and to adapt the moves applied within each region to its local structure, the procedure carries four pieces of machinery on top of the generate-evaluate-select cycle. The first partitions $\mathcal{X}_B$ into a small number of regions of architecturally similar candidates and holds one independent population in each, so that the budget of surrogate calls is divided across the regions rather than concentrated on the one that the early iterations happen to favour. The second supplies a fixed library of mutation and crossover operators, partitioned into a budget-neutral family that perturbs a candidate inside its current region and a structural family that moves it across regions, so that every offspring is feasible on construction up to repair. The third assigns to each region an independent bandit over the operators, with the operator distribution adapted across iterations from the rewards observed in that region, and additionally modulated by the parent’s local descriptor so that two parents in the same region but with different head-to-neuron balance see different operator distributions. The fourth ranks the offspring of a generation within their cohort by the surrogate gate of Section 1.4, retains the surviving fraction as the next-generation population in that region, and routes the candidates with the best gated scores across generations into a small set of archives that survive population turnover.

One iteration of the search consists of, for each region in parallel, sampling a batch of parents from the region’s current population, applying operators drawn from the region’s bandit to produce a batch of offspring, projecting the offspring back into the region under the budget repair of Section 1.6.8, evaluating the offspring on the calibration packs at the cheap fidelity (and on the full fidelity at scheduled intervals), ranking the union of offspring and incumbents within their cohort, retaining the surviving fraction as the next-generation population, and inserting the best candidates of the cohort into the global archives. The bandit and the niche-conditioned mixture are updated from the cohort scores at the end of the iteration. The cycle runs for T iterations, after which the archives and the per-region best candidates form the finalist pool. The finalist pool is re-evaluated at the highest fidelity available, with several independent resamples of the calibration set averaged, and the candidate that minimises the resampled surrogate (with a MAC tie-break inside an ϵ -front) is returned as $\hat{c}$.

Section 1.6.2 specifies the per-head and per-neuron proposal scores that the operators and the projection use to break ties between candidates of equal cost. Section 1.6.3 initialises the populations by a structure-aware sweep over coarse macro descriptors, and Section 1.6.4 defines

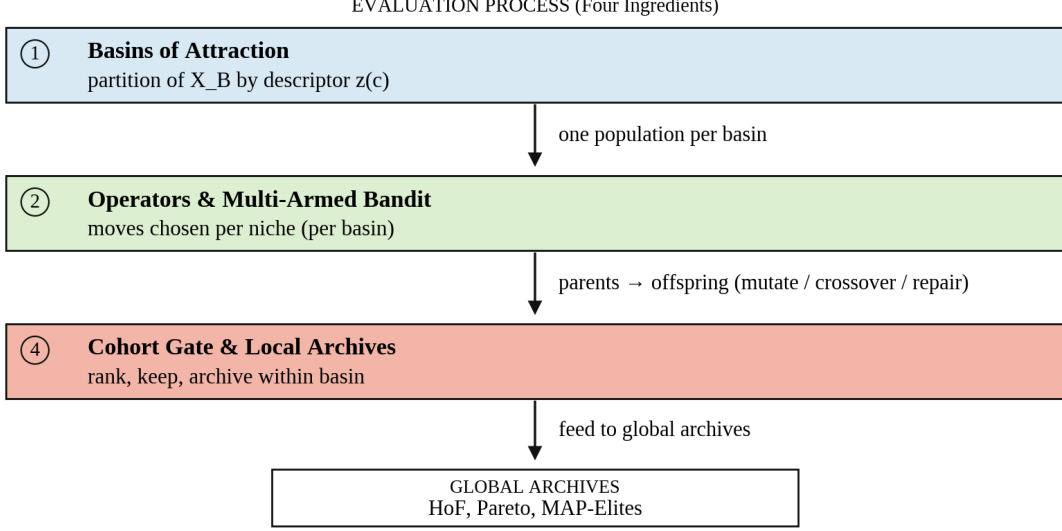


Figure 1.3: The four pieces of machinery. (1) An architectural descriptor partitions $\mathcal{X}_B$ into regions, and one population is held in each. (2) A fixed library of operators perturbs candidates within a population. (3) A bandit per region learns the operator distribution that performs best in that region. (4) Selection is by within-cohort rank under the surrogate gate of Section 1.4, and the best candidates are kept in archives that feed the final pick.

the architectural descriptor $z(c)$ and the k -means construction of the partition of $\mathcal{X}_B$ into regions. Section 1.6.5 places one population per region, with the basin spec of Section 1.6.4 restricting the masks the population is allowed to carry. Section 1.6.6 specifies the bandit, the niche-conditioned mixture, and the operator library that the bandit samples from. Section 1.6.7 describes one generation, including parent selection, the crossover variants, and the dedupe step. Section 1.6.8 defines the budget repair and the basin projection. Section 1.6.9 states the calibration fidelities. Section 1.6.10 closes the loop with the cohort gate of Section 1.4 and the archive updates. Section 1.6.12 produces $\hat{c}$.

1.6.2 Proposal Scores and Budget Manager

Before the search begins we compute per-head and per-neuron importance scores $s_H \in \mathbb{R}_{\geq 0}^{L \times H}$, $s_N \in \mathbb{R}_{\geq 0}^{L \times N}$ on the calibration set. Each entry is a local quadratic sensitivity of the teacher’s residual update at its layer to setting the corresponding unit’s gate to zero, evaluated with gradients of the teacher-forced update target. Writing $g_{\ell,h}$ for the gradient of the calibration loss with respect to head h of layer ℓ ’s gate, the head score takes the form

$$s_H[\ell, h] = \mathbb{E}_{x \sim \mathcal{D}_{\text{cal}}} [g_{\ell,h}(x)^2] + \lambda \cdot \text{MSE} \left(\tilde{\Delta}_{\ell}^{(\mathcal{T}, h=0)}, \Delta_{\ell}^{(\mathcal{T})} \right), \quad (1.31)$$

with the second term an MSE between the teacher’s per-layer update under head h zeroed and the teacher’s true update; $\lambda = 0.35$ in our default configuration. The neuron score is defined analogously over feed-forward neuron gates. Scores are row-normalised per layer to remove dependence on the absolute scale of gradients across depths.

The aggregated layer importance is

$$\text{imp}(\ell) = \sum_{h=1}^H s_H[\ell, h] + \sum_{n=1}^N s_N[\ell, n]. \quad (1.32)$$

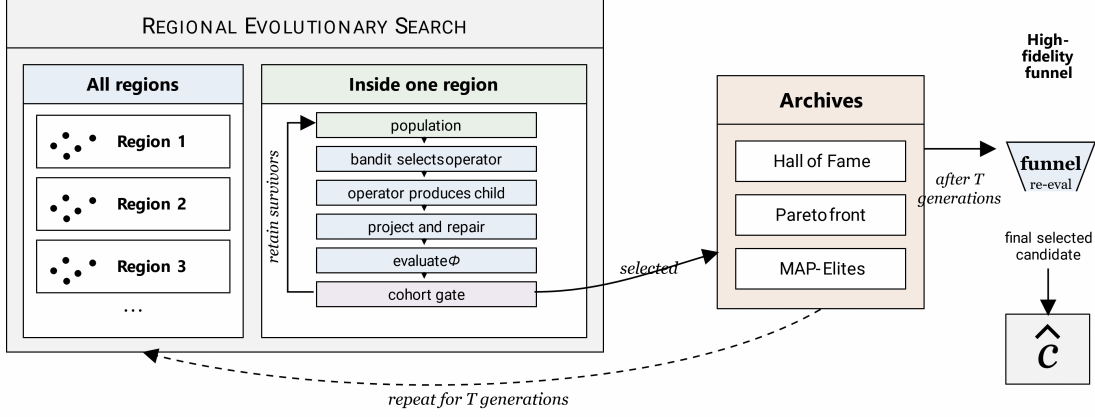


Figure 1.4: One iteration of the search and the final funnel. Each region’s population produces offspring with operators chosen by its bandit; offspring are projected back into the region, evaluated on the calibration packs, ranked by the cohort gate, and used to update the population and the archives. After T iterations, the archives and the per-region best candidates are re-evaluated at higher fidelity and one mask $\hat{c}$ is returned.

The *budget manager* maintains s_H, s_N, imp , the unit costs (c_H, c_N) , and a precomputed table of cumulative neuron-block scores $S_\ell[b] = \sum_{j=1}^b s_N[\ell, \pi_\ell(j)]$, where π_ℓ is the descending sort of $s_N[\ell, \cdot]$ and blocks are aligned to size $b_n = 16$. The table S_ℓ supports $O(1)$ marginal-gain queries: the score gained by adding the $(k+1)$ -th block of neurons to layer ℓ is $S_\ell[k+1] - S_\ell[k]$, and the score lost by removing the k -th block is $S_\ell[k] - S_\ell[k-1]$.

These proposal scores are *not* the search objective. They are priors used in three operations:

- the structure-aware sweep below selects active layers and allocates compute by them;
- the repair routine breaks ties by removing the least valuable block and adding the most valuable one;
- operator implementations sample blocks with probabilities derived from them.

The objective the cohort sees remains $\Phi(c)$ of Section 1.4.

1.6.3 Structure-Aware Sweep

The sweep seeds the search with a broad set of feasible candidates and also feeds the CVT construction of the next subsection. It is a Latin hypercube over three orthogonal coarse axes:

- $N_a \in \{L_{\min}, \dots, L_{\max}\}$: the active-layer count, with $L_{\min}$ the active-depth floor introduced above and $L_{\max} = \lfloor B/(c_H + b_n c_N) \rfloor$ the largest count whose minimum cost still fits the budget;
- $\phi_H \in \{0.05, 0.10, 0.15, 0.25, 0.35, 0.50, 0.65, 0.80, 0.92\}$: the fraction of the budget spent on attention heads, so that head-spend is $\phi_H \cdot B$ and neuron-spend is $(1 - \phi_H) \cdot B$;
- a *subset mode* for choosing the N_a active layers from the importance vector imp , taking values $\{\text{topk}, \text{noisy_topk}, \text{weighted}, \text{random}\}$, optionally paired with a partial-layer style $\{\text{mixed}, \text{attn}, \text{ffn}\}$ for ZipLM-like single-branch layers.

For one cell $(N_a, \phi_H, \text{mode})$ the sweep builds a candidate as follows. With **topk**, draw N_a indices from the multinomial defined by imp over the top- $1.75N_a$ layers; with **noisy_topk**, perturb imp by Gaussian noise of std 0.30 and take the top N_a ; with **weighted**, sample without replacement from softmax weights derived from $\text{imp} \cdot \exp(0.20 \cdot \mathcal{N}(0, 1))$; with **random**, sample uniformly. Let $A \subset \{1, \dots, L\}$ be the chosen active set. For $\ell \in A$ define per-layer weights

$$w_\ell = \text{imp}(\ell) \cdot \exp(0.35 \cdot \xi_\ell), \quad \xi_\ell \sim \mathcal{N}(0, 1), \quad \tilde{w}_\ell = w_\ell / \sum_{k \in A} w_k, \quad (1.33)$$

and allocate

$$a_\ell = \text{round}(\phi_H B \tilde{w}_\ell / c_H), \quad n_\ell = b_n \cdot \lfloor (1 - \phi_H) B \tilde{w}_\ell / (b_n c_N) \rfloor. \quad (1.34)$$

The specific heads kept in layer ℓ are sampled from a temperature-softmax of $s_H[\ell, \cdot]$ at temperature $\tau \sim \text{Unif}[0.55, 1.10]$. Layers $\ell \notin A$ keep $a_\ell = n_\ell = 0$.

Every sweep candidate is repaired into $\mathcal{X}_B$ (Section 1.6.8) and evaluated on $\mathcal{D}_{\text{cal}}^{\text{cheap}}$. The sweep yields a pool of roughly 300 feasible candidates with a controlled spread over (N_a, ϕ_H) .

1.6.4 CVT Basin Construction

The sweep gives diversity but no structure: there is no easy way to ask the search to keep working in distinct architectural regions. We obtain that structure by partitioning a behavioural descriptor space.

Descriptor. For a candidate c define

$$z(c) = (z_1, z_2, z_3, z_4, z_5, z_6) \in [0, 1]^6, \quad (1.35)$$

with the six coordinates: head MAC share $z_1 = (\sum_\ell a_\ell c_H) / \text{MAC}(c)$; neuron fraction $z_2 = (\sum_\ell n_\ell) / (LN)$; active fraction $z_3 = |\{\ell : a_\ell > 0 \wedge n_\ell > 0\}| / L$; normalised layer-budget entropy

$$z_4 = -\frac{1}{\log L} \sum_{\ell=1}^L p_\ell \log p_\ell, \quad p_\ell = \frac{a_\ell c_H + n_\ell c_N}{\text{MAC}(c)}; \quad (1.36)$$

and the head- and neuron-centre-of-mass coordinates

$$z_5 = \frac{\sum_\ell \ell \cdot a_\ell}{(L-1) \sum_\ell a_\ell}, \quad z_6 = \frac{\sum_\ell \ell \cdot n_\ell}{(L-1) \sum_\ell n_\ell}. \quad (1.37)$$

Each z_k is clamped to $[0, 1]$. The descriptor compresses an architecture into six scalars that capture how much of the budget goes to attention, how filled the feed-forward stacks are, how many layers are active, how concentrated the compute is across depth, and where in the stack heads and neurons sit.

Sample and standardise. We draw ~ 300 feasible candidates (the sweep pool plus random importance-proportional draws), evaluate $z(c_i)$, and standardise: $\bar{z} = \frac{1}{N} \sum_i z_i$, $\sigma_z = \text{std}(z_i)$, $\tilde{z}_i = (z_i - \bar{z}) / \sigma_z$.

k -means CVT. We cluster $\{\tilde{z}_i\}$ into $k = I$ cells with k -means++ initialisation (max-min seeding) and Lloyd iterations:

$$\text{label}(i) = \arg \min_c \|\tilde{z}_i - \tilde{\mu}_c\|, \quad \tilde{\mu}_c = \frac{1}{|C_c|} \sum_{i \in C_c} \tilde{z}_i, \quad (1.38)$$

iterated until labels and centres stabilise. Each cell c yields a centroid in original coordinates $\mu_c = \sigma_z \tilde{\mu}_c + \bar{z}$ and a radius r_c set to the 75th percentile of within-cell distance.

Basin specification. Each cell becomes a *BasinSpec*, a tuple

$$\mathcal{B}_c = (h_{\min}^c, h_{\max}^c, n_{\min}^c, n_{\max}^c, \phi_H \in [\phi_-^c, \phi_+^c], u \in [u_-^c, u_+^c], \rho_{\min}^c, \mu_c, r_c), \quad (1.39)$$

where $h_{\min}^c, h_{\max}^c$ are per-cell head-count bounds, $n_{\min}^c, n_{\max}^c$ are block-aligned neuron-count bounds, $[\phi_-^c, \phi_+^c]$ is a band on the head spending fraction, $[u_-^c, u_+^c]$ a band on the per-layer utilisation, and $\rho_{\min}^c$ a basin-local floor on the operation count. The bounds are derived from the 10th/35th/75th/95th percentiles of (h, n, ϕ_H) among the sweep candidates assigned to cell c , so that the basin tightly contains its members and excludes the rest.

1.6.5 Island Populations

We instantiate I islands, one per basin. Each island i is a tuple

$$\mathcal{I}_i = (\mathcal{B}_i, \text{focus}_i, \mathcal{P}_i, c_i^*, d_i^*, \text{stag}_i, \mathcal{A}_i, \text{ema}_i), \quad (1.40)$$

where $\mathcal{B}_i$ is its basin; focus_i is a label from `{head_lowN, head_midN, balanced, neuron_heavy, layer_sparse}` choosing the operator prior used to initialise its bandit; $\mathcal{P}_i$ is its population, of size P ; (c_i^*, d_i^*) is its best-ever candidate and that candidate’s distance; $\text{stag}_i \in \mathbb{N}$ is a stagnation counter; $\mathcal{A}_i$ is its EXP3-IX bandit (Section 1.6.6); and ema_i is a niche-conditioned operator EMA, defined below. The five islands cover different regions of the descriptor space, so the search is forced to maintain diversity rather than converging on a single architectural type.

Seeding. The initial population of island i is built by (i) projecting the sweep winner into $\mathcal{B}_i$, (ii) calling a basin-targeted seed factory $P/2$ times to produce candidates whose (h, n, ϕ_H) already sit inside $\mathcal{B}_i$, (iii) ensuring all active-layer counts in $[L_{\min}, L_{\max}]$ appear at least once, and (iv) optionally injecting a small number of partial-layer candidates (attn-only or fn-only) for the `layer_sparse` basin. All seeds are then run through $\Pi_{\mathcal{B}_i}$, the basin projection of Section 1.6.8.

1.6.6 Per-Island Bandit Operators

Each island runs an independent multi-armed bandit over the operator set $\mathcal{O}$, with $|\mathcal{O}| = 26$ operators partitioned into a budget-neutral micro pool $\mathcal{O}_\mu$ and a structural macro pool $\mathcal{O}_M$ (Section 1.7). We use the EXP3 algorithm of Auer et al. (2002) with the implicit-exploration (EXP3-IX) correction of Neu (2015). Let $K = |\mathcal{O}|$, let $w_i[o] \in \mathbb{R}_{>0}$ be the weight of operator o on island i , and let $\gamma \in (0, 1)$ be the exploration parameter (default $\gamma = 0.07$). The sampling distribution is

$$p_i[o] = (1 - \gamma) \cdot \frac{w_i[o]}{\sum_{o' \in \mathcal{O}} w_i[o']} + \frac{\gamma}{K}, \quad (1.41)$$

i.e. a γ -mixture of exploitation (proportional to weights) and uniform exploration. When the bandit chooses operator o with sampled probability $\hat{p}$ and the operator produces a reward $r_o \in [0, 1]$, the EXP3-IX update is

$$\hat{r}_o = \frac{r_o}{\hat{p} + \gamma}, \quad w_i[o] \leftarrow w_i[o] \cdot \exp\left(\frac{\gamma \hat{r}_o}{K}\right), \quad (1.42)$$

with periodic rescaling of $\{w_i[o]\}$ when $\max_o w_i[o]$ exceeds 10^6 . The denominator $\hat{p} + \gamma$ instead of $\hat{p}$ alone bounds the variance of $\hat{r}_o$ even when an operator has been driven to very low probability, which avoids the score blow-ups EXP3 is known to exhibit on adversarial regret traces.

Island priors. The initial weight vector is set by focus_i . For example, the `layer_sparse` focus initialises the macro operators `drop_layer` at weight 3.0, `drop_multi` at 2.5, `layer_rewire` at 2.0, and the others uniformly at 1.0, biasing early generations of that island toward depth pruning. The `equilibrium` focus initialises `budget_neutral_swap_head` and `budget_neutral_swap_neurons` at 2.0 each, biasing toward fine reallocation. Five other foci cover the remaining basins.

Niche-conditioned operator EMA. Within each island we also maintain an EMA of the reward each operator has produced, conditioned on the parent’s discrete niche

$$\text{niche}(c) = (N_a, \text{mac_bucket}, \text{shape_bucket}), \quad \text{mac_bucket} \in \{\text{low}, \text{mid}, \text{high}\}, \quad (1.43)$$

with `mac_bucket` = low if $\text{MAC}(c)/B < 0.80$, mid if < 0.93 , high otherwise; and `shape_bucket` classifying c as `head_heavy`, `balanced`, or `neuron_heavy` by the sign of the head-utilisation

minus neuron-utilisation gap. When the bandit samples an operator for a parent whose niche is ν , the base probabilities $p_i[o]$ of (1.41) are scaled by the EMA bonus

$$\tilde{p}_i[o; \nu] \propto p_i[o] \cdot (0.5 + \text{ema}_i[\nu, o]), \quad \text{ema}_i[\nu, o] \in [0, 1], \quad (1.44)$$

and renormalised. This adapts the operator distribution to the part of the architecture space the parent currently sits in, without sacrificing the exploration floor that γ guarantees on the base bandit.

1.6.7 Generation Step

We now describe the per-generation loop. The pseudocode in Algorithm 1 is the per-island work that runs in each generation t for each island $\mathcal{I}_i$; islands run in parallel and synchronise at archive updates and migration.

Cheap rank and fitness sharing. We first compute the cheap distance $\Phi^{\text{cheap}}(c)$ for every $c \in \mathcal{P}_i$ and convert to a rank-normalised score $f_j = 1 - r_j/|\mathcal{P}_i|$, where r_j is the position of c_j in increasing distance order. The rank scores are then shared on the structural distance matrix $D \in \mathbb{R}^{P \times P}$ between population members:

$$\tilde{f}_j = \frac{f_j}{1 + \sum_{k \neq j} \max(0, 1 - D_{jk}/\sigma_s)}, \quad (1.45)$$

where D_{jk} is a normalised Hamming-like distance on the mask of c_j and c_k and σ_s (default 0.20) is the sharing radius. A candidate that sits in a dense cluster sees its score divided by an effective crowding count; isolated candidates keep their score unmodified.

Parents and elites. The top $e \approx \lceil \text{elite_size}/I \rceil$ candidates by $\tilde{f}$ are copied unchanged as elites. The remaining $P - e$ slots are filled by offspring whose parents are chosen by tournament-with-novelty. We build the candidate pool from the top half of $\mathcal{P}_i$ by $\tilde{f}$. For each parent slot $k = 1, \dots, P/2$ we sample 4 contenders from the pool and pick the one maximising

$$\text{score}_k(j) = \tilde{f}_j + 0.20 \cdot \min_{k' < k} D_{j,p_{k'}} + 0.01 \cdot u_j, \quad u_j \sim \text{Unif}[0, 1], \quad (1.46)$$

where $p_1, \dots, p_{k-1}$ are the parents already chosen. The novelty term $\min_{k' < k} D_{j,p_{k'}}$ pushes successive parents away from each other in mask space.

Operator schedule. We split the $P - e$ offspring slots into a macro fraction $\beta_M(t) = \min(0.80, 0.35 + 0.03 \cdot \text{stag}_i)$ and a micro fraction $1 - \beta_M(t)$. Macro offspring are drawn from the bandit restricted to $\mathcal{O}_M$, with crossover probability $p_X = 0.65$; micro offspring are drawn from the bandit restricted to $\mathcal{O}_\mu$, with $p_X = 0.25$; with no operator-mode restriction the default $p_X = 0.40$. The crossover-or-mutate decision is per offspring.

Crossover. Three forms are mixed by rolling $r \sim \text{Unif}[0, 1]$:

- $r < 0.45$: *topology-budget* crossover. One parent contributes the active set and the head pattern in each active layer; the other contributes the neuron counts. Active layers of the topology parent are paired with active layers of the budget parent by a random permutation; the budget parent's neuron count at the paired layer is copied (clipped to $[b_n, N]$).
- $0.45 \leq r < 0.75$: *single-cut* layer-wise swap. Choose a cut point $\ell^* \in \{1, \dots, L - 1\}$ uniformly; the child takes layers $1, \dots, \ell^*$ from p_1 and layers $\ell^* + 1, \dots, L$ from p_2 .
- $r \geq 0.75$: *uniform* layer-wise swap. For each ℓ independently, with probability $1/2$ replace layer ℓ of the child by layer ℓ of the second parent.

The result is then either passed to a mutation operator (with probability $1 - p_X$ the child is a clone of the chosen base parent and goes directly to mutation) or moved on to projection.

Mutation. The sampled operator o is applied to the child. Section 1.7 defines all 26 operators; the cohort gate of Section 1.6.10 provides the binary reward used to update $\mathcal{A}_i$.

Dedupe and basin probes. Offspring are deduplicated first by exact mask signature, then greedily by structural near-duplication at radius $\sigma_d = \max(0.06, 0.3\sigma_s)$: a new candidate c is dropped if any already-kept candidate has $D(c, c') < \sigma_d$. On full-evaluation generations the procedure additionally injects up to 8 basin-local probes around c_i^* (specifically, random budget-neutral swaps from c_i^* projected into $\mathcal{B}_i$) to keep an exploration floor near the island’s incumbent.

Algorithm 1: One generation on island $\mathcal{I}_i$.

Input: Island $\mathcal{I}_i$ at generation t ; population $\mathcal{P}_i$ of size P ; bandit $\mathcal{A}_i$.

Output: Updated population $\mathcal{P}_i^{(t+1)}$; archives; updated bandit and EMA.

- 1 $\{\Phi^{\text{cheap}}(c)\}_{c \in \mathcal{P}_i} \leftarrow$ cached cheap-pack eval
 - 2 $\{\tilde{f}_j\} \leftarrow$ rank-normalise then fitness-share via (1.45)
 - 3 $E \leftarrow$ top e by $\tilde{f}$; parents $\leftarrow$ tournament-with-novelty $P/2$ times via (1.46)
 - 4 **for** $k = 1, \dots, P - e$ **do**
 - 5 mode $\leftarrow$ **macro** with prob $\beta_M(t)$, else **micro**
 - 6 $p^* \leftarrow$ random parent; $\nu \leftarrow$ niche(p^*); sample $o \sim \tilde{p}_i[\cdot; \nu]$ via (1.44)
 - 7 $c \leftarrow$ crossover(p^* , mate) with prob $p_X(\text{mode})$ else clone of p^*
 - 8 $c \leftarrow o(c)$; $c \leftarrow \Pi_{\mathcal{B}_i}(c)$ (Section 1.6.8)
 - 9 push c to offspring pool if not duplicate of any prior offspring
 - 10 **end**
 - 11 $\mathcal{P}_i^{(t+1)} \leftarrow E \cup \text{dedupe}(\text{offspring})$, truncated to P
 - 12 cheap-evaluate offspring; on every k -th generation, full-evaluate top half
 - 13 compute cohort scores via (1.28); reward operators; update $\mathcal{A}_i$ via (1.42) and ema_i
 - 14 update HoF, Pareto archive, MAP-Elites
-

1.6.8 Candidate Projection and Repair

Mutation and crossover do not preserve $\mathcal{X}_B$ or the basin bounds. A projection step $\Pi_{\mathcal{B}}$ enforces both. Concretely, the procedure of Frantar and Alistarh (2022, Section 6.2) for budget repair is generalised to also enforce basin-range constraints.

Pass 1: protected core. Identify the top $L_{\min}$ layers by $\text{imp}(\ell)$ among the candidate’s active layers and mark them *protected*. Force each protected layer to carry at least one head and at least b_n neurons. Unprotected layers are left as-is, including partial states with only heads or only neurons.

Pass 2: budget band. While $\text{MAC}(c) > (1 + \varepsilon)B$, remove the block whose removal loses the smallest score from the cumulative table S_ℓ , breaking ties by lowest layer importance. While $\text{MAC}(c) < (1 - \varepsilon)B$, add the block whose addition gains the largest score, restricted to layers in the basin’s allowed set. Each iteration is $O(\log L)$ on a heap of marginal gains.

Pass 3: head-count bounds. Enforce $h_{\min}^c \leq \sum_\ell a_\ell \leq h_{\max}^c$, $n_{\min}^c \leq \sum_\ell n_\ell \leq n_{\max}^c$, and $\phi_-^c \leq \phi_H(c) \leq \phi_+^c$ by multinomial fill/drain. The fill probability over a set of addable layers is

$$q_\ell \propto \text{softmax}(\text{inv}(a_\ell) \cdot \tau), \quad \text{inv}(x) = 1 / \max(x, 1), \quad (1.47)$$

biasing fills toward currently-low-count layers, with $\tau \sim \text{Unif}[0.8, 1.3]$. The drain distribution is symmetric. Both are applied iteratively until the bounds hold.

Pass 4: CVT centroid repair. If $\|z(c) - \mu_c\| > r_c \cdot 0.18$, apply up to 14 unit-step moves that decrease $\|z(c) - \mu_c\|$: each step adds or removes one head or one block of neurons in the layer that most improves the descriptor distance, subject to the basin and budget bounds remaining feasible. The repair stops as soon as the candidate enters the centroid tolerance or no descending unit move exists.

The composition of these four passes is the basin projection $\Pi_B(c)$. The non-basin variant used outside basin contexts (e.g. for finalists) omits Pass 4 and uses the union of all basins' bounds in Pass 3.

1.6.9 Cheap Screening and Full Evaluation

Cheap evaluation (Section 1.5) is run on every offspring at every generation. Full evaluation is run on the top half of cheap-screened offspring every k generations (default $k = 2$). Distances are cached by $(\text{exact_sig}(c), \text{fidelity}, R)$, so a candidate seen again between generations or between islands is not re-evaluated; near-duplicates inside the dedupe radius σ_d collapse onto the same key when they share an exact mask.

1.6.10 Cohort Selection and Archive Updates

The cohort $\mathcal{C} = \mathcal{P}_i^{\text{offspring}} \cup E$ is ranked by the gate of Section 1.4: candidates with trajectory rank $r^{\text{traj}} \leq \kappa$ are ordered by task rank with an $\mathcal{O}(10^{-3})$ trajectory tie-break, and the rest are pushed strictly below by adding 1 to r^{traj} (Equation (1.28)).

Reward. The bandit reward for the operator o that produced c is the binary indicator $r_o = \mathbf{1}[\Phi^{\text{gate}}(c) < \Phi^{\text{gate}}(\text{parent})]$, with a small bonus added if c improves c_i^* . Equation (1.42) then updates $\mathcal{A}_i$. The niche EMA $\text{ema}_i[\nu, o]$ is updated by the exponentially smoothed reward

$$\text{ema}_i[\nu, o] \leftarrow (1 - \eta_{\text{ema}}) \cdot \text{ema}_i[\nu, o] + \eta_{\text{ema}} \cdot r_o, \quad \eta_{\text{ema}} = 0.10. \quad (1.48)$$

Archives. Three global archives are maintained.

- *Hall of Fame* $\mathcal{H}$: bounded sorted list of size 50, keyed by exact mask signature, with a cap of 3 entries per macro signature (defined as the active-layer set together with rounded ϕ_H). Insertion is by smallest distance; eviction is of the largest distance within the same macro key first, then the global largest.
- *Pareto archive* $\mathcal{P}$: non-dominated set on $(\Phi(c), \text{MAC}(c))$ of size ≤ 100 . A new point (c, d, m) dominates an old (c', d', m') iff $d \leq d' \wedge m \leq m'$ with strict inequality in one coordinate.
- *MAP-Elites* $\mathcal{M}$: grid archive on $(N_a, \text{round}(\phi_H, 1))$, keeping the best-distance candidate per cell.

Updates to all three happen at the cohort-gate step.

1.6.11 Stagnation, Restarts, and Migration

Each island maintains a stagnation counter stag_i that is incremented every full-evaluation generation on which c_i^* does not improve, and reset to zero whenever it does.

Restart. When $\text{stag}_i \geq 20$, the island restarts. Half of the new population is drawn from the Hall of Fame, filtered to candidates whose basin mismatch penalty

$$\rho(c, \mathcal{B}_i) = \mathbf{1}[c \notin \mathcal{B}_i] \cdot (c_h \mathbf{1}[h(c) \notin [h_{\min}^i, h_{\max}^i]] + c_n \mathbf{1}[n(c) \notin [n_{\min}^i, n_{\max}^i]] + c_\phi \mathbf{1}[\phi_H(c) \notin [\phi_-^i, \phi_+^i]]) \quad (1.49)$$

is below 0.6; the rest are fresh basin seeds. Every restart candidate is projected into $\mathcal{B}_i$. The bandit is reset to its focus $_i$ prior (Section 1.6.6) and the cosine temperature schedule is warm-restarted. The stagnation counter goes back to zero.

Migration. Every T_{mig} generations (default 10), islands are connected by a ring topology and exchange a small number of candidates. Island i sends $m \approx 5$ of its top-half members to island $j = (i + 1) \bmod I$; the receiving island projects them into its own basin and replaces m of its bottom-half slots. Migration mixes the search states without collapsing the basin assignments, because every migrant is projected through $\Pi_{\mathcal{B}_j}$ on arrival.

Diversity injection. If the global pool ever drops to ≤ 2 unique exact signatures, a third of every island is replaced by budget-neutral perturbations of randomly chosen current members, generated by 3–8 steps of `budget_neutral_swap` operators.

1.6.12 Final Funnel Selection

At the end of the T generations, the search collects a finalist set

$$\mathcal{F} = \{\hat{c}^{(T)}\} \cup \bigcup_i \text{top}_3(\mathcal{P}_i) \cup \text{sample}_5(\mathcal{M}) \cup \text{sample}_5(\mathcal{H}) \cup \text{sample}_5(\mathcal{P}), \quad (1.50)$$

deduplicated by exact signature. Each $c \in \mathcal{F}$ is re-evaluated at full fidelity averaged over R_{final} independent full-pack resamples (default $R_{\text{final}} = 4$):

$$\Phi^{\text{final}}(c) = \frac{1}{R_{\text{final}}} \sum_{r=1}^{R_{\text{final}}} \Phi(c; \mathcal{D}_{\text{cal}}^{\text{full},r}). \quad (1.51)$$

Let $d^* = \min_{c \in \mathcal{F}} \Phi^{\text{final}}(c)$. We define the ϵ -front

$$\mathcal{F}_\epsilon = \{c \in \mathcal{F} : \Phi^{\text{final}}(c) \leq d^* + \epsilon\}, \quad (1.52)$$

with ϵ a small absolute slack (default $\epsilon = 0.005$), then sort $\mathcal{F}_\epsilon$ lexicographically by $(\Phi^{\text{final}}, -\text{MAC})$ and return the top of the sort as

$$\hat{c} \in \arg \min_{c \in \mathcal{F}_\epsilon} (\Phi^{\text{final}}(c), -\text{MAC}(c)). \quad (1.53)$$

The ϵ -front breaks the practically meaningless ties between architectures whose full-fidelity surrogate distances are within the calibration noise of one another, and the secondary key $-\text{MAC}$ resolves them in favour of the architecture using more of its allowed compute. The returned candidate $\hat{c}$ is the input to the recovery procedure of Section 1.10.

1.7 Candidate Generation and Mutation Operators

The mutation operators are the local moves by which the evolutionary algorithm explores the feasible set. Their design determines what neighbourhoods of the search space the algorithm can reach. This section describes each operator individually. For each we state what the operator takes as input, what it changes, how the result is brought back into the feasibility band, what kind of move it implements in the macro signature space, and what failure mode justifies its existence.

We organise the operators into four families. The first family contains budget neutral swaps that move structures within the feasibility band without changing the total compute. The second family contains growth and shrinkage operators that change a specific axis at the cost of compensating elsewhere. The third family contains layer level operators that activate or deactivate entire layers. The fourth family contains macroscale operators that rearrange the compute allocation across layers.

1.7.1 Family One: Budget Neutral Swaps

The budget neutral family preserves the compute cost up to one repair step. These operators are the workhorses of the search because they keep the candidate inside the feasibility band by construction and therefore explore allocation patterns at fixed compute.

Budget neutral head swap. The operator picks two layers, removes one head from the first, and adds one head to the second. The change in compute is zero up to the per layer rounding. The operator is justified by the observation that the trajectory term often rewards reallocating attention compute from a layer with poor delta agreement to a layer with better delta agreement, at zero cost.

Budget neutral neuron swap. The same idea applied to a single block of neurons. One neuron block is removed from one layer and added to another. The operator handles the more common case in which a layer’s feed forward width is the binding constraint on its trajectory match.

Budget neutral head for neurons. A cross axis swap. The operator removes some number of heads from one layer and adds the corresponding compute as neuron blocks to another, possibly the same layer. The compute exchange ratio is determined by the cost constants c_H/c_N , which for our base encoder is approximately the ratio of one head to four neuron blocks. The operator is the only one in this family that can change the head fraction of the macro signature, and it is essential for exploring the head versus neuron trade off.

Budget neutral neurons for head. The symmetric operator, removing neuron blocks and adding heads. Both directions of cross axis swap are necessary because the bandit reward favours one direction or the other depending on the current population.

1.7.2 Family Two: Growth and Shrinkage With Compensation

The growth and shrinkage operators change the count of one structure type and compensate by changing another, so that the total compute returns to the band after repair.

Grow heads, shrink neurons elsewhere. The operator picks a target layer, adds one head to it, and removes neuron blocks from other layers chosen by least importance until the budget band is restored. The operator differs from the cross axis swap in that the growth and the compensation are not in the same layer, so the operator can concentrate compute in a head heavy layer while distributing the cost across the rest of the network.

Grow neurons, shrink heads elsewhere. The symmetric operator on the neuron axis.

Shrink heads, free budget for later operators. A purely subtractive operator that removes one head from a layer without immediately compensating. The resulting candidate is below the budget band and is then repaired by adding the most useful block elsewhere. The operator’s purpose is to liberate budget from a low value head, so that subsequent moves can place the freed compute where the importance scores suggest.

Shrink neurons, free budget. The symmetric operator on the neuron axis.

1.7.3 Family Three: Layer Level Operators

These operators act on whole layers and effect macroscopic changes in the architecture.

Drop layer. The operator picks a layer that currently has at least one surviving structure and removes all of them. The resulting candidate has one more fully dropped layer. The freed compute is then redistributed to other layers by the repair step. The operator is the only one that increases the number of fully dropped layers, and it is essential for the search to reach architectures of low effective depth.

Activate layer. The symmetric operator. The operator picks a fully dropped layer and reactivates it by placing a minimum block of heads or neurons, depending on a random choice. The freed compute is taken from the lowest value structures elsewhere. The operator is necessary to escape from architectures with too many dropped layers, since the search is otherwise prone to monotonically dropping layers without ever restoring them.

Activate attention only. A specialised variant of the activate operator. The reactivated layer is given only attention heads, with zero neurons. The result is a half skip layer of the kind described in Section 1.2. The operator exists because attention only layers occupy a distinct region of the macro signature space and the bandit otherwise has difficulty reaching them.

Activate feed forward only. The symmetric variant. The reactivated layer has only neuron blocks, no heads.

1.7.4 Family Four: Macroscale Rearrangements

These operators are not budget neutral on a single layer but redistribute the global compute allocation by larger amounts.

Macro budget shift. The operator picks a contiguous block of two or three layers and redistributes the compute among them according to the per layer importance scores. Heads and neurons are moved together. The operator is the closest analogue to a gradient step on the global compute allocation.

Neuron rebalance. The operator equalises the neuron counts across the active layers, subject to the budget. The result is a candidate whose feed forward width is approximately uniform. The operator is the antidote to architectures that the search has driven to a single fat layer at the expense of all others.

Layer rewire. The operator permutes the per layer compute allocations while preserving the multiset of allocations. Concretely, if the current architecture has the per layer compute pattern $(c_1, c_2, \dots, c_L)$, the operator returns a candidate with pattern $(c_{\pi(1)}, c_{\pi(2)}, \dots, c_{\pi(L)})$ for some permutation π . The operator does not change the macro signature but moves the candidate in the per layer fine grained space. It is useful when the metric is sensitive to which layers are fat and which are thin, and the global allocation is otherwise reasonable.

Equilibrium mutate. A multi step operator that applies a short sequence of small swaps to bring the candidate to a local equilibrium of the importance based budget allocator. The result is a candidate whose per layer compute is proportional to the per layer importance score sums. The operator’s role is to act as a reset when an island has explored a non productive region.

Jitter. A small perturbation that randomly increments or decrements the head and neuron counts by one or two units in a small number of layers. The operator generates close neighbours of the parent and is useful when the bandit has learned that the current population is near a local optimum.

Group reallocate. The operator picks two groups of layers, computes their total compute, and redistributes the total across the groups according to the importance scores. The operator differs from the macro budget shift in that the groups are not contiguous, so the operator can implement non local moves in the compute allocation.

Sparsity shift. The operator chooses uniformly between increasing and decreasing the candidate’s head fraction by a small amount, with neurons compensating. The operator’s role is to explore the head versus neuron trade off when the bandit has been stuck on a single fraction for several generations.

Head neuron trade. A simple operator that picks a layer, removes one head, and adds the equivalent neuron compute as blocks. The operator is the per layer analogue of the budget neutral cross axis swap.

Sweep grow. The operator picks the most important layer that currently has fewer than the maximum compute and adds compute to it until the budget band is restored. The operator is biased toward concentrating compute in already strong layers and is the antidote to architectures that are too uniform.

1.8 Population Management, Islands, and Diversity

The population structure of the evolutionary search is not a flat one. The population is partitioned into islands, each island occupies a region of a low dimensional descriptor space, each island has its own bandit over operators, and a Hall of Fame and a Pareto archive track the globally best candidates across islands. This section explains the diversity preserving machinery and the rationale for each component.

1.8.1 Why the Population Is Not Flat

The simplest possible evolutionary algorithm uses a single population, evaluates all candidates against a single objective, and replaces the worst by mutated copies of the best. Empirically, on the search landscape we have described, this design collapses the population onto a narrow region of macro signatures within a few generations. The reason is that the trajectory term can favour certain depth allocations, and the bandit amplifies that preference once a local region starts improving.

The collapse is not a bug per se. If the macro signature that the metric prefers is also the one with the highest post recovery accuracy, the collapse is the correct behaviour. The problem is that we have no a priori guarantee that the metric’s preferred macro signature matches the post recovery optimum, and the collapse forecloses the possibility of finding out.

Diversity preservation is therefore a methodological commitment. The population is structured so that exploration of distinct macro topologies continues throughout the search, regardless of which region the local optimum of the metric lies in.

1.8.2 Centroidal Voronoi Tessellation Islands

The islands partition the population by a Centroidal Voronoi Tessellation of a six dimensional descriptor space. The descriptor of a candidate is the tuple

$$\phi(c) = (\phi_1, \phi_2, \phi_3, \phi_4, \phi_5, \phi_6) \in [0, 1]^6, \quad (1.54)$$

where the components are the share of MAC spent on heads, the active layer fraction, the entropy of the per layer MAC distribution, the centre of mass of heads along the depth axis, the centre of mass of neurons along the depth axis, and the neuron fraction.

The tessellation has I cells, each defined by a centroid in $[0, 1]^6$. A candidate is assigned to the cell of the nearest centroid, with ties broken uniformly. The centroids are computed once at the start of the search by Lloyd iteration on a random sample of feasible candidates, so that the cell volumes are approximately equal.

Each cell corresponds to one island. The island has a bounded size, typically twenty four candidates, and a fixed budget of evaluations per generation. The candidates within an island compete with each other for survival under the cohort selection rule.

1.8.3 Bandit Per Island

Each island maintains its own multi armed bandit over the set of mutation operators of Section 1.7. The per island bandit allows different niches to favour different operators. An island in a region of high active layer fractions may favour the drop layer operator, while an island in a region of low active layer fractions may favour the activate layer operator.

The bandit update is an exponential weight scheme with a small exploration term. Concretely, after observing rewards $r_1, \dots, r_K$ for operator k across the offspring of generation t , the operator weight is updated as

$$w_k \leftarrow w_k \cdot \exp(\eta \cdot \bar{r}_k), \quad (1.55)$$

with $\bar{r}_k$ the mean observed reward and η the learning rate. The weights are renormalised to sum to one, and a uniform mixture of weight γ is added so that no operator's probability falls below γ/K . The exploration term γ is set to a small constant in $[0.01, 0.10]$ and prevents the bandit from concentrating prematurely on a single operator.

1.8.4 Migration Between Islands

Periodically the algorithm performs a migration. The best individual of each island is copied to a neighbouring island, defined as the nearest centroid in the descriptor space, and replaces the worst individual of that island. Migration occurs every M generations, with M a small integer such as ten.

Migration has two effects. It propagates good candidates across niches, so that a discovery in one niche can seed exploration in a neighbouring niche. It also injects diversity into islands whose population has converged onto a narrow region within the niche, since the migrated candidate is likely to be different from the local best.

1.8.5 Hall of Fame

The algorithm maintains a global Hall of Fame of fixed capacity, typically fifty entries. The Hall of Fame stores the candidates with the best cohort selection scores ever observed across all islands and all generations. A candidate is admitted to the Hall of Fame if its score is better than the worst current entry. Entries are deduplicated by their exact mask signature.

The Hall of Fame plays two roles. First, it acts as an elitist memory that prevents the search from forgetting good candidates that have been replaced in their island's population. Second, it serves as the pool of candidates from which the finalists are drawn at the end of the search.

1.8.6 Pareto Archive

A second global structure tracks the non dominated frontier of the population in a two dimensional space of cohort selection score and compute cost. Even though the search operates under a hard compute budget, the cost varies slightly within the feasibility band, and the Pareto archive surfaces the candidates that achieve the best score at each cost level within the band.

The Pareto archive has bounded capacity, and admission is based on non domination rather than on absolute score. A candidate is admitted if no current entry strictly dominates it. When

the archive is full, the most crowded entry is evicted to maintain capacity, where crowding is measured by the volume of the dominated rectangle in the score and cost plane.

The Pareto archive supplements the Hall of Fame because the latter is purely score based and does not distinguish between two candidates with similar scores but different costs.

1.8.7 Macro Signature Uniqueness

A simple measure of diversity that the algorithm tracks is the number of unique macro signatures in the population. The macro signature is the triple of total heads, total neurons, and active layer count from Section 1.2. A population with many unique macro signatures is exploring broadly, and a population with few unique macro signatures has converged onto a narrow region.

The unique macro signature count is reported every generation in the search log. A persistent fall in the count is the canonical indicator of premature convergence and would in principle trigger a restart, although our default configuration does not include an automatic restart mechanism.

1.8.8 Stagnation Counters and Restart

Each island maintains a stagnation counter that increments by one every generation in which the island fails to produce a new best. The stagnation counter resets when a new best is produced. The counter is reported in the search log alongside the generation summary.

A restart rule is available in the implementation but disabled by default. The rule resets an island’s population to a random subset of the Hall of Fame when its stagnation counter exceeds a threshold. The restart is intended to escape from a region of the descriptor space that the island has exhausted, but it has the side effect of losing the operator weights that the island’s bandit has learned, and we found in practice that the cost outweighed the benefit on our calibration set sizes.

1.8.9 Why Diversity Matters Under a Noisy Metric

The diversity preserving machinery is motivated by a general property of search under a noisy objective. When the objective is the true post recovery accuracy, the question of diversity reduces to the question of escaping local optima. When the objective is a noisy proxy for the post recovery accuracy, diversity is additionally a hedge against the noise itself. A flat population collapsed onto the proxy’s optimum stops exploring entirely, and if the proxy’s optimum is not the true optimum the population has no way to discover that fact. A diverse population continues to evaluate candidates from other macro topologies, and any of those evaluations may reveal a region of the space where the proxy is unexpectedly low or where its prediction is unexpectedly accurate.

Given the observed gap between surrogate score and recovered accuracy, the cost of diversity preservation is a worthwhile hedge. The search ablations compare this design against reduced variants that remove the island structure.

1.9 Alternative Search Baselines

The evolutionary search of Section 1.6 is a complex object, and a reader is entitled to ask whether the complexity is necessary. This section describes two baseline search procedures that operate on the same surrogate metric, the same calibration set, and the same compute budget as the evolutionary search. The purpose of the baselines is to isolate the contribution of the evolutionary machinery from the contribution of the surrogate metric itself.

1.9.1 Baseline One: Random Search

The first baseline draws a fixed number of candidates uniformly from the feasible set and returns the candidate with the lowest surrogate score on the full calibration pack.

The sampling procedure works as follows. A random Latin hypercube configuration is drawn, in the same form as the bootstrap of Section 1.6, namely an active layer count, a head fraction, and a subset mode. The configuration is converted to a candidate by the importance based allocation procedure, and the candidate is repaired into the feasibility band. The result is added to the pool of evaluated candidates. The procedure repeats until the total number of forward passes matches the compute budget allocated to the evolutionary search.

The random search uses the same calibration packs and the same surrogate metric as the evolutionary search. The two procedures therefore differ only in how candidates are proposed.

The random search is the strongest possible baseline against the claim that the evolutionary machinery adds value over the metric. If the random search achieves comparable recovered accuracy at comparable compute, the evolutionary machinery is superfluous. If the random search achieves lower recovered accuracy, the difference is the value added by the evolutionary machinery.

1.9.2 Baseline Two: Beam Search

The second baseline replaces the population structure of the evolutionary search with a beam of fixed width and replaces the bandit operator selection with a uniform sampling of operators.

The procedure works as follows. A beam of B candidates is initialised by the Latin hypercube bootstrap. At each iteration, each candidate in the beam is expanded by applying a fixed number of mutation operators sampled uniformly at random from the operator set of Section 1.7. The resulting offspring are scored on the cheap surrogate, and the top B candidates across the union of the previous beam and the new offspring are retained. The procedure repeats for a fixed number of iterations.

The beam search is intermediate in complexity between the random search and the evolutionary search. It uses the same operators as the evolutionary search but lacks the island structure, the bandit, and the cohort relative ranking.

1.9.3 Fairness Constraints

The comparison of the three procedures is meaningful only if they operate under identical resource budgets. We enforce four constraints.

First, the total number of full forward passes is the same across the three procedures, equal to the product of the evolutionary search’s per generation population size and number of generations.

Second, the calibration set is fixed across the three procedures, including the cheap and full packs and the resamples.

Third, the budget constraint and the feasibility band are the same.

Fourth, the post search recovery procedure is the same. The chosen architecture of each baseline is passed through the multi phase recovery of the search sectionsI with identical hyperparameters, and the validation accuracy of the recovered model is reported.

1.9.4 What the Comparison Shows

The comparison is reported with the experimental results. We summarise here the qualitative role of each baseline.

The random search achieves a lower recovered accuracy than the evolutionary search at the same compute budget. The gap widens at higher sparsity. The reason is that the feasible set at high sparsity contains a small fraction of architectures with non degenerate recoverability, and

random sampling reaches that small fraction with probability proportional to its measure. The evolutionary search reaches the small fraction faster because the bandit and the cohort selection bias the search toward architectures whose local moves improve the surrogate.

The beam search achieves a recovered accuracy intermediate between the random search and the evolutionary search. The beam search inherits the locality of the operators but does not benefit from the diversity preservation of the island structure or from the operator weighting of the bandit. Its main failure mode is premature convergence onto a narrow region of the macro signature space.

The qualitative pattern is consistent with the methodological claim that the evolutionary machinery is not redundant on top of the surrogate metric, and that the search procedure contributes a measurable fraction of the methodology’s empirical advantage.

1.9.5 Why Not Compare Against Continuous Relaxation Methods

A reader may ask why we do not include a continuous relaxation method as a baseline, for example a method that trains the masks as continuous variables under a sparsity regulariser. The reason is that such a method is not a search at all in the sense used in this report. It is a parameter training procedure with mask training built in, and the recovered model is the output of the same training that produces the masks.

The comparison of a continuous relaxation method against the methodology described here is meaningful but it would require equalising training compute across the two methods, which is a substantially more involved experimental design than the head to head search baselines of this section. We treat the comparison as out of scope and refer the reader to the published comparisons in [Xia et al. \(2022\)](#) and [Kurtic et al. \(2023\)](#) for the relative merits of the two paradigms.

1.10 Rescale Recovery Distillation

The search returns a fixed pruned architecture $\hat{c} = (\mathbf{M}^H, \mathbf{M}^N)$. At high sparsity, this architecture usually requires parameter recovery before evaluation: the surviving heads and neurons inherit the teacher parameters, whereas the residual pathway is altered by the removed components. Recovery therefore trains the selected masked student while keeping the architecture fixed.

The recovery objective combines representation-level and prediction-level supervision. These objectives impose distinct constraints on the compressed model. Hidden-state matching aligns the internal trajectory with the teacher computation, whereas label and logit supervision align the final predictive distribution. Under severe compression, the available capacity may be insufficient to optimise both objectives effectively from the initial step. The recovery procedure is therefore staged. The first stage uses intermediate representation matching to stabilise the surviving layers. The second stage shifts the optimisation target to prediction-level distillation and task supervision. This follows the general principle of knowledge distillation ([Hinton et al., 2015](#)) and the use of intermediate supervision in compact language models ([Jiao et al., 2020](#)).

Figure 1.5 summarises this recovery logic. It places the fixed mask before all optimisation steps, then separates internal representation recovery from final prediction recovery. The diagram also shows the role of EMA as an evaluation mechanism, so the training objectives remain limited to the two staged losses defined below.

The left-to-right order in Figure 1.5 fixes the implementation order used by Rescale. The student parameters are initialised from the teacher. The head and neuron masks are hardened to binary tensors and are not optimised. Fully dropped layers are bypassed as identities during masked training. The classifier is frozen during the representation stage and unfrozen for prediction recovery. The procedure therefore separates architecture selection from parameter recovery: search chooses the structure, and Rescale trains the selected structure.

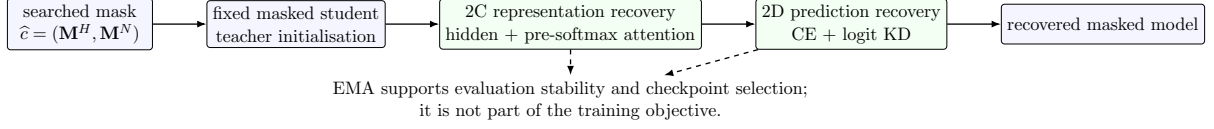


Figure 1.5: Recovery flow after architecture search. The mask is fixed throughout recovery; only the surviving parameters are trained.

1.10.1 2C: Intermediate Representation Recovery

The first recovery stage trains the masked student before the classifier is allowed to adapt. Its purpose is to restore an internal trajectory that can support subsequent prediction training. If prediction loss is applied immediately, classifier gradients are propagated through representations that may already be poorly aligned with the teacher. Hidden-state supervision gives the surviving layers a direct target and supplies a denser signal than the task label alone.

Let the teacher hidden states be

$$h_0^{(\mathcal{T})}(x), h_1^{(\mathcal{T})}(x), \dots, h_L^{(\mathcal{T})}(x),$$

where $h_0^{(\mathcal{T})}$ is the embedding output. Let $\mathcal{S}(\hat{c})$ be the ordered set of active student layers under the selected mask. A fixed set of teacher anchor depths $\mathcal{A}$ is chosen along the teacher depth. The mapping

$$\pi : \mathcal{S}(\hat{c}) \rightarrow \mathcal{A}$$

assigns each active student layer to a teacher anchor according to its relative depth. This mapping is dynamic because the active student layers depend on the searched architecture. A shallow candidate and a deeper candidate therefore use different anchor assignments.

For a batch $\mathcal{B}$ and a padding mask $m_x(p)$, the hidden-state loss is

$$\mathcal{L}_{\text{hid}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{\sum_p m_x(p) \|h_{s,p}^{(\mathcal{S})}(x; \theta) - h_{a,p}^{(\mathcal{T})}(x)\|_2^2}{d \sum_p m_x(p)} \right], \quad (1.56)$$

where $\mathcal{M} = \{(s, \pi(s)) : s \in \mathcal{S}(\hat{c})\}$.

Hidden states constrain the output of each matched layer. The stage also uses pre-softmax attention supervision to constrain how the surviving attention heads form their token-to-token interactions. Let $A_{a,r}^{(\mathcal{T})}(x)$ denote the pre-softmax attention score matrix of teacher anchor layer a and head r , before the row softmax is applied. Let $A_{s,r}^{(\mathcal{S})}(x; \theta)$ be the corresponding student score for a surviving head. With $\mathcal{H}_s$ denoting the active heads in student layer s , the attention-score loss is

$$\mathcal{L}_{\text{attn}}(\theta) = \frac{1}{|\mathcal{M}|} \sum_{(s,a) \in \mathcal{M}} \mathbb{E}_{x \in \mathcal{B}} \left[\frac{1}{|\mathcal{H}_s|} \sum_{r \in \mathcal{H}_s} \frac{\|P_x(A_{s,r}^{(\mathcal{S})}(x; \theta) - A_{a,\rho(r)}^{(\mathcal{T})}(x)) P_x\|_F^2}{\left(\sum_p m_x(p)\right)^2} \right], \quad (1.57)$$

where P_x masks padding positions and ρ maps each surviving student head to its teacher head index. The loss is pre-softmax because the score matrix contains the geometry that the softmax later normalises; matching probabilities alone can hide large score shifts when the distribution is already sharp.

The 2C objective is

$$\mathcal{L}_{2C}(\theta; t) = \lambda_h(t) \mathcal{L}_{\text{hid}}(\theta) + \lambda_a(t) \mathcal{L}_{\text{attn}}(\theta). \quad (1.58)$$

The hidden term is the primary recovery signal. The attention term is assigned a smaller weight because it constrains the mechanism that produces the layer update. The weights may be fixed

or scheduled during the stage. In both cases, the stage remains representation-facing because it excludes task cross entropy and logit distillation.

During this stage, the classifier is frozen. The checkpoint path tracks development accuracy, and an exponential moving average of the student parameters is maintained for evaluation stability:

$$\bar{\theta}_t = \mu \bar{\theta}_{t-1} + (1 - \mu) \theta_t. \quad (1.59)$$

EMA is a standard parameter averaging device (Polyak and Juditsky, 1992). In this procedure, it is used for evaluation stability and checkpoint selection. It does not modify the loss function or introduce an additional supervision source.

1.10.2 2D: Prediction-Level Distillation

After 2C, the masked student has been trained toward the teacher’s internal trajectory. The classifier is then unfrozen and the objective shifts to prediction behaviour. This stage uses two signals: the ground-truth task labels and the teacher’s pre-softmax logits. The labels anchor the model to the dataset, whereas the teacher logits provide soft class relations that are absent from one-hot labels.

Let $z^{(\mathcal{T})}(x)$ and $z^{(\mathcal{S})}(x; \theta)$ be the teacher and student logits. With label smoothing parameter ϵ , the smoothed target distribution for a class label y is

$$q_\epsilon(c | y) = (1 - \epsilon) \mathbf{1}[c = y] + \frac{\epsilon}{C}.$$

The task loss is

$$\mathcal{L}_{\text{ce}}(\theta) = -\mathbb{E}_{(x,y) \in \mathcal{D}_{\text{train}}} \sum_{c=1}^C q_\epsilon(c | y) \log \text{softmax}(z^{(\mathcal{S})}(x; \theta))_c. \quad (1.60)$$

The predictive distillation term uses temperature-scaled teacher logits:

$$\mathcal{L}_{\text{kd}}(\theta) = T^2 \mathbb{E}_{x \in \mathcal{D}_{\text{train}}} \text{KL} \left(\text{softmax} \left(\frac{z^{(\mathcal{T})}(x)}{T} \right) \parallel \text{softmax} \left(\frac{z^{(\mathcal{S})}(x; \theta)}{T} \right) \right). \quad (1.61)$$

The temperature exposes non-argmax class information, and the factor T^2 keeps the gradient scale comparable as T changes (Hinton et al., 2015). If confidence weighting is enabled, each example’s KD term is multiplied by

$$w_{\mathcal{T}}(x) = \max_c \text{softmax}(z^{(\mathcal{T})}(x))_c,$$

so uncertain teacher predictions contribute less to the distillation signal.

The 2D objective is a dynamically weighted multi-objective loss:

$$\mathcal{L}_{2D}(\theta; t) = \alpha(t) \mathcal{L}_{\text{ce}}(\theta) + (1 - \alpha(t)) \mathcal{L}_{\text{kd}}(\theta), \quad (1.62)$$

where $\alpha(t)$ increases slowly over the stage. Early optimisation assigns more weight to KD because the teacher distribution provides smoother gradients while the classifier adapts to the recovered representations. Later optimisation assigns more weight to CE because the final model is evaluated against task labels rather than teacher predictions.

The optimisation flow is therefore

$$\begin{aligned} \text{fixed mask} &\rightarrow \text{2C representation recovery} \\ &\rightarrow \text{classifier unfreeze} \\ &\rightarrow \text{2D CE/KD recovery} \\ &\rightarrow \text{best raw or EMA checkpoint.} \end{aligned} \quad (1.63)$$

The architecture remains fixed across the entire flow. Rescale can improve the surviving parameters, layer normalisations, and classifier weights. It cannot add heads, add neurons, or change where the search placed capacity. The recovered accuracy is therefore a joint outcome of the searched architecture and the staged distillation procedure.

1.11 Experimental Protocol

This section fixes the experimental setting. The protocol keeps the task data, teacher model, search budget, recovery procedure, and reporting format constant, so differences in accuracy can be attributed to the pruning method rather than to changes in infrastructure.

1.11.1 Tasks

The experiments use two GLUE classification tasks (Wang et al., 2018). QNLI is a binary question-sentence inference task derived from SQuAD (Rajpurkar et al., 2016). MNLI is a three-way natural language inference task with matched and mismatched genres (Williams et al., 2018). We use the public training splits for search and recovery data, and the public development splits for validation. The test splits are not used. These two tasks give different stress cases: QNLI is binary and smaller, whereas MNLI is larger and has three labels.

1.11.2 Teacher and Student

The teacher is the task-fine-tuned uncased base model introduced by Devlin et al. (2019), built on the Transformer encoder of Vaswani et al. (2017). It has twelve layers, hidden dimension 768, twelve attention heads per layer, and feed forward width 3072. The student starts from the teacher weights and applies the head and neuron masks defined in Section 1.2. Search evaluates masked students without adding new modules. Recovery then trains the masked student with the distillation procedure described in Section 1.10.

1.11.3 Reproducibility Setting

Unless a variance study is explicitly reported, each run uses seed zero. When variance is reported, seeds $\{0, 1, 2\}$ are used and the table gives the mean with standard error. Runs are executed on a single GPU workstation. Mixed precision is used where stable (Micikevicius et al., 2018); the surrogate metric is evaluated in full precision because it compares small residual-update differences.

1.11.4 Search and Recovery Budget

The search uses the calibration protocol of Section 1.5 and the evolutionary procedure of Section 1.6. Candidate costs are computed with the operation model of Section 1.3, and infeasible candidates are repaired or discarded by the projection of Section 1.6.8. This keeps all search methods under the same target compute band.

After search, the selected masks are recovered with intermediate-representation distillation and prediction distillation. The recovery stage follows the distillation principle of Hinton et al. (2015) and the intermediate matching used in TinyBERT (Jiao et al., 2020). Optimisation uses AdamW (Loshchilov and Hutter, 2019) with warmup and cosine decay (Loshchilov and Hutter, 2017).

1.11.5 Baselines and Reporting

All baselines use the same teacher, calibration data, target budget, and recovery procedure. The comparison includes random search, beam search, and structured pruning references from

CoFi (Xia et al., 2022) and ZipLM (Kurtic et al., 2023). We report development accuracy, giga operations at reference length 128, compute keep ratio, active heads, active neurons, and active layers. MNLI uses the matched development split unless stated otherwise. Differences within one standard error are treated as ties.

1.12 Conclusion

The report has described a methodology for structured pruning of a transformer encoder, applied to text classification tasks at high sparsity. The methodology combines a guided evolutionary architecture search under a hard compute budget, a surrogate distance metric scored on a small calibration set, and a multi phase distillation recovery procedure that restores task accuracy on the chosen architecture.

The conclusions the experimental evidence supports are modest. The methodology produces recovered accuracies competitive with the structured pruning literature at compute keep ratios between zero point three and zero point zero five on Question Natural Language Inference and Multi Genre Natural Language Inference. The evolutionary machinery contributes a measurable improvement over random and beam baselines. The cohort selection rule is the most consequential single design choice in the search, and the two phase structure is necessary for the recovery. The surrogate metric is a useful but imperfect predictor of recovered accuracy, with a Spearman rank correlation in absolute value around zero point five on the diagnostic set.

The methodology has known limitations that the report has documented. The surrogate’s task term saturates at high sparsity. The trajectory term has a magnitude bias. The recovery procedure has known failure modes in its exponential moving average selection and in its train versus deploy gap at half skip layers. The architectures the search produces at the highest sparsity drop layers and starve the late layers in patterns that reflect both the structure of the surrogate metric and the structure of the task.

Future work that the present analysis points to includes a per layer reweighting of the trajectory term to compensate for its magnitude bias, a brief adaptation stage before the surrogate is computed to bring it closer to a measurement of recoverability, a modification of the deployment substitution rules to preserve layer normalisations at half skip layers, and an extension of the methodology to non classification tasks. Each of these directions is testable by a controlled experiment whose protocol the report’s experimental sections provide a template for.

The methodology is, in our view, a reasonable point in the design space of structured pruning procedures. It is not a final answer. The components are individually well understood, the failure modes are documented, and the improvements remaining to be tested are concrete enough to be the subject of a follow up study.

Bibliography

- Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM J. Comput.*, 32(1):48–77, 2002.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- Elias Frantar and Dan Alistarh. SPDY: Accurate pruning with speedup guarantees. In *ICML*, 2022.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NeurIPS Deep Learning Workshop*, 2015.
- Xiaoqi Jiao et al. TinyBERT: Distilling BERT for natural language understanding. In *Findings of EMNLP*, 2020.
- Eldar Kurtic, Elias Frantar, and Dan Alistarh. ZipLM: Inference-aware structured pruning of language models. In *NeurIPS*, 2023.
- Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *NeurIPS*, 2019.
- Paulius Micikevicius et al. Mixed precision training. 2018.
- Gergely Neu. Explore no more: Improved high-probability regret bounds for non-stochastic bandits. In *NeurIPS*, 2015.
- Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *EMNLP*, 2016.
- Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *EMNLP Workshop BlackboxNLP*, 2018.
- Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL-HLT*, 2018.
- Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. In *ACL*, 2022.